**Ejercicio práctico núm. 08**

**Lenguaje JAVA SE: "Factoría"**

**Empleo de las características esenciales de la programación orientada a objetos**

En esta práctica trabajaremos tres mecanismos fundamentales en la programación orientada a objetos:

- **Herencia** y **polimorfismo**, dos de los mecanismos que caracterizan de manera exclusiva en la programación orientada a objeto. También está la **encapsulación**.

- Las asociaciones entre clases. Pese a que la implementación de este tipo de relaciones ya se empezó a trabajar en los proyectos previos, en esta práctica profundizaremos en los aspectos de su programación.

- Tratamiento de errores y excepciones. Este mecanismo formará parte de la gestión de errores de ejecución que sucedan durante el funcionamiento de la aplicación.

## 1. INTRODUCCIÓN

Una pequeña fábrica comercial (`Factory`), nos ha encargado una aplicación para la gestión de sus trabajadores (`Workers`). En esta `Factory` existen varios tipos de `Workers`:

De una parte, tenemos los vendedores/as (`Salesman`), que se encargan de hacer el máximo número de ventas, puesto que reciben una pequeña comisión por cada venta.

Los ejecutivos/as (`Executive`) se dedican a vigilar el correcto funcionamiento de su área (es decir, organizar sus `Salesman`, indican qué productos se venderán, donde se situarán, etc...).

Finalmente, cada `Executive` tiene un secretario/a (`Secretary`) que se encarga de las cuestiones jurídicas y administrativas.

Por ahora, y con la intención de ampliar el proyecto en un futuro muy próximo, nos han pedido que nos centremos en el cálculo de los salarios (`salary`) de los `Workers`.

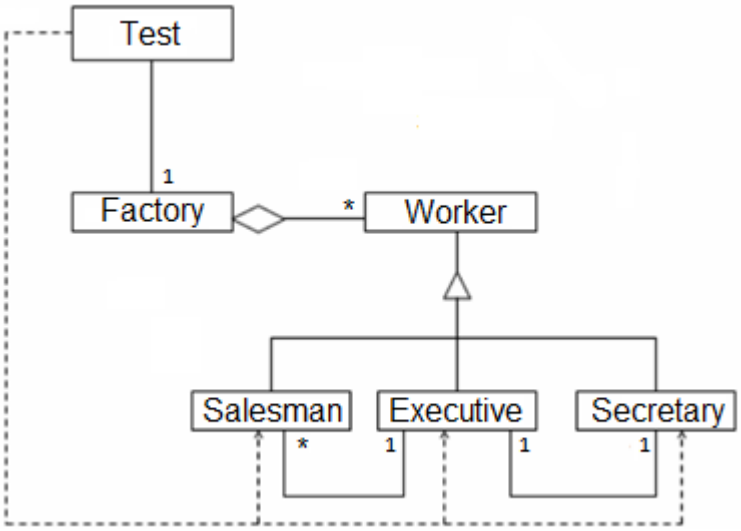
El `salary` de un `Secretary` es fijo (1.200€); los `salary` de los `Salesman`, en cambio, con objeto de motivarlos para vender el mayor número de productos, consta de una parte fija (1.300€) más unas comisiones del importe vendido (5%).

Finalmente tenemos los `Executive`, que cobran un tanto fijo (2.500€) más una comisión de todas las ventas hechas por los `Salesman` que están a su cargo (2%).

A fin de mes, el contable de la `Factory` desea obtener un listado de todos los `Workers` de la `Factory`, con su `dni` y `salary` a cobrar, a fin y efecto de ingresarlo a su cuenta corriente.

Como se prevé que aparezcan nuevos tipos de `Workers`, se quiere que el sistema de listados comentado anteriormente sea suficientemente genérico como para no tener que reimplementar la gestión del listado de `salary`s cada vez que se añadan nuevos tipos de `Worker`s y que, al mismo tiempo, este listado los incluya.

## 2. MODELO ESTÁTICO




## 3. DISEÑO DE CLASES A IMPLEMENTAR

Las clases que se especifican en este apartado son las únicas que se han de implementar.

Por cada clase se han de implementar los atributos y métodos que se describen, sin cambiar el nombre, ni los parámetros, ni el tipo de su valor de retorno, puesto que estos constituyen la interfaz que usan las clases de soporte que se proporcionan. Esto no quiere decir que estos métodos y atributos sean los únicos que puedan tener vuestras clases, puesto que, seguramente, si se aplica un buen diseño descendente, aparecerán nuevos métodos o atributos privados que se habrán de añadir a estas clases.

### 3.1 Clase Worker

| *Worker* |
| --- |
| `-name:String`<br>`-address:String`<br>`-dni:String`<br>`-salary:double` |

```
+Worker(pName:String, pAddress:String, pDni:String)
+getName():String
+getDni():String
+getAddress():String
+toString():String
+equals(obj:Object):boolean
+computeSalary():double
+getSalary():double
+setSalary(salary:double)
```

Esta clase representa un `Worker`, con su nombre (`name`), dirección (`address`), número de DNI (`dni`), un `salary` base y una manera de calcular el `salary` del `Worker`. Esta clase es una clase abstracta.

Por simplificar, si el resultado del cálculo de la comisión que cobran algunos workers es un número con decimales, entonces se ha de redondear por defecto (hacia abajo).

**Atributos**:

`-name:String` Atributo que guarda el nombre del `Worker`.
`-address:String` Atributo que guarda la dirección del `Worker`.
`-dni:String` Atributo que guarda el `dni` del `Worker`.
`-salary:double` Atributo que guarda el `salary` base del `Worker`.

## Métodos:

`+Worker(pName:String, pAddress:String, pDni:String)`
Constructor de la clase. Asigna el `name`, `address` y `dni` pasados como parámetros.
`+getName():String`
Accesor de lectura, que nos da el `name` del `Worker`.
`+getDni():String`
Accesor de lectura, que nos da el `dni` del `Worker`.
`+getAddress():String`
Accesor de lectura, que nos da la `address` del `Worker`.
`+toString():String`
Devuelve una cadena de caracteres que contiene el `name`, el `dni` y address del `Worker` separados entre ellos por comas.
`+equals(obj:Object):boolean`
Método comparador. Retorna true si el objeto pasado por parámetro es igual al actual. Se considera que dos `workers` son iguales si tienen el mismo número de `dni`.
`+computeSalary():double`
Método abstracto. Se utiliza para calcular el `salary` específico del `Worker` (depende del tipo de Worker y, por lo tanto, hace falta que lo implemente cada subclase de `Worker`).
`+getSalary():double`
Accesor de lectura para el atributo del `salary` base del `worker`.
`+setSalary(salary:double)`
Accesor de escritura para el atributo del `salary` base del `worker`.

Pensando en la reutilización, podría ser interesante definir un método *accesor* de escritura para el atributo `address`, dado que es fácil que una persona cambie de `address` con cierta regularidad. Sin embargo, no tiene sentido hacer esto con los otros atributos, puesto que las personas no cambian de `name` ni de número de `dni`.

Suponiendo que cada persona tiene un número de `dni` propio y diferente del de otra persona, éste es el atributo que se ha decidido utilizar para saber si un trabajador es igual o no a otro.

Por tanto, el método `equals`, que sirve para comparar trabajadores, hará una comparación entre los `dni` de estas personas para saber si son la misma.

Aunque esta clase es abstracta (no puede haber instancias de la misma), nos resulta útil como generalización de los diferentes tipos de `workers` de la `Factory` y, además, nos permitirá, en un futuro, definir nuevos tipos de `workers` por especialización de la misma.

Esta clase `Worker` dispone del método `computeSalary` que, implementado en cada subclase, proporcionará el `salary` que debe percibir el `worker` en el mes en curso.

### 3.2 Clase Salesman

| Salesman |
|---|
| -salesMonth:double<br>-COMISSION:double<br>-SALARYBASE:double<br>-executive:Executive |
| +Salesman(pName:String, pAddress:String, pDni:String)<br>+computeSalary():double<br>+newSale(pSale:double)<br>+getSalesMonth():double<br>+changeMonth()<br>+setExecutive(pExecutive:Executive)<br>+getExecutive():Executive<br>+hasExecutive():boolean |

Esta clase representa un tipo de `Worker`, los `Salesman`, con un `salary` base fijo y un método de cálculo de su `salary`.

**Atributos:**

`-salesMonth:double`

El importe de las ventas hechas por el `Salesman` durante este mes.

`-COMISSION:double`

Constante que representa la comisión de un `Salesman`.

`-SALARYBASE:double`

Constante que representa el `salary` base de un `Salesman`.

`-executive: Executive`

Atributo que indica cuál es el `Executive` responsable del `Salesman`.

**Métodos:**
`+Salesman(pName:String, pAddress:String, pDni:String)`
Constructor de la clase `Salesman`. Crea un `Salesman` con su `name`, `address` y `dni`.
`+computeSalary():double`
Método utilizado para calcular el `salary` específico del `Worker`(`salesman`) a través del polimorfismo.
`+newSale(pSale:double)`
Método que incrementa el importe del total vendido con el importe de la nueva venta.
`+getSalesMonth():double`
Accesor de lectura del importe del total de ventas efectuadas por el `Salesman` en el mes actual.
`+changeMonth()`
Método que indica que se ha cambiado de mes; por lo tanto se debe poner a cero las ventas hechas en este mes.
`+setExecutive(pExecut:Executive)`
Accesor de escritura del atributo `executive`.
`+getExecutive():Executive`
Accesor de lectura del atributo `executive`.
`+hasExecutive():boolean`
Método que indica si el `Salesman` tiene `Executive` asignado.

La constante `COMISSION` toma el valor 5, puesto que un salesman tiene una comisión del 5% respeto a las ventas que ha hecho. Se ha definido también la constante `SALARYBASE`, que representa el `salary` base de un `salesman` como 1300€. Se debe asignar este valor al atributo `salary` en el constructor de la clase. El cálculo del `salary` de un `salesman` (método `computeSalary`) se hace calculando el 5% de las ventas (redondeando hacia abajo) y sumándolo al `salary`.

### 3.3 Clase Executive

| Executive |
|---|
| -COMISSION:double<br>-SALARYBASE:double<br>-salesmen:Salesman*<br>-secretary:Secretary |
| +Executive(pName:String, pAddress:String, pDni:String)<br>+computeSalary():double<br>+addSalesman(pNewSalesman:Salesman)<br>+setSecretary(pNewSecretary:Secretary)<br>+listWorkers():String |

Esta clase representa un tipo de Worker, el executive, con un salary propio y un método de cálculo del mismo concreto.

Atributos:
`-COMISSION:double`
Constante que representa la comisión de un Executive.
`-SALARYBASE:double`

Constante que representa el salary base de un executive.
-salesmen:Salesman*
Lista de Salesman del Executive.
-secretary:Secretary
Secretary del Executive.

**Métodos:**

+Executive(pNname:String, pAddress:String, pDni:String)
Constructor de la clase Executive. Crea un Executive con un dni, name y address concretos.
+computeSalary():double
Método utilizado para calcular el salary específico del worker (executive) a través del polimorfismo.
+addSalesman(pNewSalesman:Salesman)
Método que añade un nuevo salesman como salesman del executive, si y sólo si, este salesman no tiene un Executive asignado.
+setSecretary(pNewSecretary:Secretary)
Accesor de escritura. Método que asigna pNewSecretary como el nuevo Secretary del executive, si y sólo si, este Secretary no tiene un Executive asignado.
+ listWorkers():String
Método que devuelve un String que contiene el listado de todos los Salesmen del Executive.

La constante COMISSION toma el valor 2 debido a que un Executive tiene una comisión del 2% respeto al importe total de las ventas que han hecho los Salesman que tiene a su cargo.

Se define también la constante SALARYBASE, que representa el salary base de un Executive, 2500€. Se asigna este valor al atributo salary en el constructor de la clase.

La lista de Salesman que un Executive tiene a su cargo (atributo salesmen) se debe representar mediante una colección de tamaño variable, puesto que no podemos saber, a priori, cuál es el número máximo de Salesman que un Executive puede tener. El cálculo del salary de un Executive se hace calculando el 2% de las ventas (redondeando hacia abajo) de todos sus Salesman y sumándolo al salary.

El método addSalesman debe causar una excepción OverrideException en caso de que el Salesman a añadir ya tenga Executive.
El método setSecretary debe causar la misma excepción en caso de que el Secretary a asignar ya tenga Executive.

### 3.4 Clase Secretary

| Secretary |
| --- |
| -executive:Executive<br>-SALARYBASE:double |

| |
|---|
| +Secretary(pNname:String, pAddress:String, pDni:String)<br>+computeSalary():double<br>+setExecutive(pExecutive:Executive)<br>+getExecutive():Executive<br>+hasExecutive():boolean |

Esta clase representa un tipo de Workers , los Secretarys, con un salary base propio y un método de cálculo del salary.

Atributos:
-executive: Executive
Atributo que indica cuál es el Executive del Secretary.
-SALARYBASE: double
Constante que representa el salary base de un Secretary.

Métodos:

+Secretary(pNname:String, pAddress:String, pDni:String)

Constructor de la clase Secretary. Crea un Secretary con un name, address y dni.
+computeSalary()

Método utilizado para calcular el salary específico del Worker (Secretary) a través del polimorfismo.
+setExecutive(pExecutive:Executive)

Accesor de escritura del atributo Executive.
+getExecutive():Executive

Accesor de lectura del atributo executive.
+hasExecutive():boolean

Método que indica si el Secretary tiene Executive.

Se ha definido la constante SALARYBASE, que representa el salary base de un Secretary como 1200€. Se asigna este valor al atributo salary base en el constructor de la clase. El cálculo del salary de un Secretary (método computeSalary) consiste, simplemente, en devolver el valor del atributo salary base del Secretary.

### 3.5 Clase Factory

| Factory |
|---|
| -name:String<br>-workers:Worker* |

```
+Factory()
+Factory(pName: String)
+setName(pName:String)
+getName():String
+addWorker(pNewWorker: Worker)
+getWorkers():Worker*
+getWorker(pDni:String): Worker
+listSalaryWorkers():String
+toString():String
```

Esta clase representa una `Factory` con un `name` y una lista de `workers`.

**Atributos:**

`-name:String`
Indica el nombre de la `Factory`
`-workers:Worker*`
Lista de `Workers` de la `Factory`

**Métodos:**

`+Factory()`
Constructor vacío. Crea una `Factory` sin nombre concreto, pero inicializa la lista de `workers`
`+Factory(pName:String)`
Constructor que crea una `Factory` con un nombre concreto e inicializa la lista de `Workers`.
`+setName(pName:String)`
Accesor de escritura del nombre de la `Factory`.
`+getName():String`
Accesor de lectura del nombre de la `Factory`.
`+addWorker(pNewWorker:Worker)`
Método que añade un nuevo `Worker` a la `Factory`, si y sólo si, el `Worker` no estaba dado de alta en la `Factory`.
`+getWorkers():Worker*`
Accesor de lectura de los `Workers` de la `Factory`.
`+getWorker(pDni:String): Worker`
Accesor de lectura de un `Worker` concreto de la `Factory`.
`+listSalaryWorkers():String`
Método que construye un String con el listado de todos los `Workers` de la `Factory` y el `salary` que deben cobrar.
`+toString():String`
Devuelve un texto con el nombre y datos de la empresa para el listado

Es destacable la presencia de dos constructores sobrecargados. Uno sirve para crear la `Factory` sin un nombre concreto, y el otro para crearla con un nombre concreto. Se puede ver también que el constructor con parámetros puede llamar al que no los tiene, a través de la instrucción `this()`.

El método addWorker lanzará una excepción creada a tal efecto, OverrideException, en caso de que el Worker a añadir ya sea un worker de la Factory.

El método getWorker lanza una excepción NotFoundException en caso de que no exista, dentro de la Factory, ningun Worker que tenga el número de DNI pasado como parámetro a este método.

En el método listSalaryWorkers es especialmente importante el hecho de que se trabaja con independencia de la tipología de Workers que hay en la Factory. Es decir, que debe hacer un recorrido por todos los Workers de la Factory y, para cada uno de ellos, hacer una conversión de tipo a Worker y llamar al método computeSalary (por polimorfismo, se llamará al método computeSalary específico de cada tipo de Worker). De este modo, la implementación de listSalaryWorkers no deberá cambiar si se añaden nuevos tipos de Workers en la Factory.

El mismo razonamiento anterior es también aplicable al método getWorker, dónde también se trabaja a nivel de Worker para abstraerse de los diferentes tipos de Workers.

## 4. CLASES ADICIONALES

- Junto con el enunciado se entrega una clase Test (al final de este documento) que prueba el funcionamiento correcto del programa y los correspondientes juegos de prueba. Esta clase comprueba las siguientes funcionalidades del programa:
- Insertar Secretary
- Insertar un Salesman
- Insertar Executive
- Asignar Secretary a Executive
- Añadir Salesman a Executive
- Añadir una venta (sale) a un Salesman
- Mostrar Worker
- Listar el salary de todos los Workers de la Factory
- Listar los Workers de un Executive

Además hay que gestionar excepciones para controlar los siguientes errores:

**0. Worker No Existente** (NotFoundException)
Se debe lanzar cuando se haga referencia a un worker que no exista. Por ejemplo: en el método getWorker(pDni:String):Worker
de la clase Factory, se debe lanzar esta excepción en el supuesto de que ningún worker de la factory tenga el pDni pedido.

**1. Worker de otro Executive** (OverrideException)
Se debe causar cuando se intente insertar en un Executive un Worker que ya estaba asignado a otro Executive. Por ejemplo: en el método addSalesman

pNewSalesman:Salesman) de la clase Executive, se debe lanzar esta excepción en el caso de que el salesman a insertar ya trabaje para otro Executive. Tener en cuenta que un Secretary también es un Worker.

El método listWorkers() de la clase Executive debe seguir el siguiente formato:

```
nameSecretary DNISecretary addressSecretary <SaltoDeLínea>
nameSalesman_1 DNISalesman addressSalesman_1 <SaltoDeLínea>
nameSalesman_2 DNISalesman addressSalesman_2 <SaltoDeLínea>
... ...
nameSalesman_N DNISalesman_N addressSalesman_N <SaltoDeLínea>
```

Esta lista ha de estar ordenada por orden de inserción. Por ejemplo, el primer salesman será el primero que se ha asignado al executive, etc.

Para listar el salary de todos los Workers de la Factory, hará falta llamar al método listSalaryWorkers() de la clase Factory, que devuelve un String. A fin y efecto de que la salida vuestra sea igual a la nuestra, debéis seguir el siguiente formato:

```
nameWorker_1 DNIWorker_1 salaryWorker_1 <SaltoDeLínea>
nameWorker_2 DNIWorker_2 salaryWorker_2 <SaltoDeLínea>
... ... ...
nameWorker_N DNIWorker_N salaryWorker_N <SaltoDeLínea>
```

Esta lista ha de estar ordenada por orden de inserción. Por ejemplo, el primer Worker será el primero que se ha insertado en la Factory , etc.

## 5. FASES

**Fase 1**: Implementación de las clases correspondientes a la herencia: Worker, Salesman, Executive y Secretary. Prueba de creación de Worker, Salesman, Executive y Secretary.

**Fase 2**: Implementar las relaciones Salesman/Executive, Executive/Secretary y Factory/Worker, aplicándolo a:

- Insertar Secretary
- Insertar Salesman
- Insertar Executive
- Añadir nueva venta a Salesman
- Listar los Workers de un Executive

**Fase 3**: Listar el salary de todos los workers de la Factory y mostrar worker.
Aplicación correcta del polimorfismo en todos los métodos implementados.
**Fase 4**: Gestión de excepciones.

## 6. FORMATO DE ENTREGA

La entrega consiste en un único fichero .zip que, cuando se descomprima, tenga la siguiente estructura:

- Un directorio de nombre src/ dónde estarán únicamente los .java correspondientes al programa.
- Un directorio que contenga todos los ficheros generados por el javadoc procedentes de las clases mediante el comando javadoc -d ./doc ./src/*.java (automatizado desde NetBeans – *dist/javadoc*- o Eclipse).
- En el directorio principal ha de haber un fichero denominado README.TXT dónde se explicará en formato de texto plano hasta dónde habéis implementado la práctica, si habéis tenido problemas, qué os falla y todos los comentarios que creáis necesarios.
- Se adjuntará el archivo resultante .jar correspondiente a la solución de entrega al cliente.

El nombre del archivo comprimido zip debe ser `ApellidosNombre-practica8-POO.zip` donde: *ApellidosNombre* son vuestros apellidos y nombre, respectivamente.

## 7. CRITERIOS DE EVALUACIÓN

A la hora de corregir la práctica se tendrán en cuenta los siguientes factores:

1. La práctica compila. Para optar a la valoración de la práctica es imprescindible que compile a la primera
2. La ejecución no se interrumpe en ningún momento.
3. La salida de la información por parte de la clase Test es la esperada.
4. Se aplican los principios SOLID de la programación orientada a objetos, así como la: encapsulación, herencia y polimorfismo.
5. Hace un uso correcto de las clases de la API: Colecciones, comparaciones, utilidades, etc.
6. La documentación de las clases es correcta.
7. Limpieza y claridad del código. Uso de buenas prácticas.

Para optar a una valoración mínima de aprobado deben cumplirse correctamente al menos los 3 primeros criterios

```java
package es.factoria;

public class Test {

    public static void main(String[] args) throws OverrideException {
        // Crear la Factory
        Factory laEmpresa = new Factory("ALCOA no-se-cierra");
        // Crear e insertar Secretary
        Secretary secretary1 = new Secretary("Juan", "Gran Vía, 22" ,"1234404J");
        // Crear e insertar Salesman
        Salesman salesman1 = new Salesman("Pedro", "Alcalá, 620", "1324324M");
        Salesman salesman2 = new Salesman("Montse", "Carlos Martín Álvarez, 77",
"8859486I");
        // Crear e insertar Executive
        Executive executive1 = new Executive("Marcos", "Serrano, 50", "5326374V");
        // Añadir venta a Salesman
        salesman1.newSale(500);
        // Asignar Salesman a Executive
        try {
            executive1.addSalesman(salesman1);
            executive1.addSalesman(salesman2);
            executive1.addSalesman(salesman1); // Error deliberado
        }
        catch(OverrideException ovx){
            System.out.println("ERROR -> " + ovx.getMessage());
        }
        try { // Asignar Secretary a Executive
            executive1.setSecretary(secretary1);
            executive1.setSecretary(secretary1); // Error deliberado
        }
        catch(OverrideException ovx){
            System.out.println("ERROR -> " + ovx.getMessage());
        }
        System.out.println("Trabajadores del executive: " + executive1);
        // Listar los workers de un executive
        System.out.println(executive1.listWorkers());
        // Añadir workers a la factory
        laEmpresa.addWorker(secretary1);
        laEmpresa.addWorker(executive1);
        laEmpresa.addWorker(salesman1);
        laEmpresa.addWorker(salesman2);
        System.out.println("Salarios de la factory: " + laEmpresa);
        System.out.println(laEmpresa.listSalaryWorkers());
        salesman1.newSale(3324); // Añadir venta a salesman
        System.out.println("Salarios de la factory: " + laEmpresa);
        System.out.println(laEmpresa.listSalaryWorkers());
        try {
            System.out.println("Worker con dni: 1324324M -> " +
laEmpresa.getWorker("1324324M"));
            System.out.println("Worker con dni: 1324325M -> " +
laEmpresa.getWorker("1324325M")); // Error deliberado: no existe ese dni...
        }
        catch(NotFoundException nfe){
            System.out.println("ERROR -> " + nfe.getMessage());
        }
    }
}
```